

Dynamic Many-Light Importance Sampling for Real-Time Ray Tracing

Pedro da Silva Freire¹^a
pedro.silva.freire@tecnico.ulisboa.pt

João Madeiras Pereira¹^b
jap@inesc-id.pt

¹ INESC-ID, Instituto Superior Técnico, University of Lisbon, Rua Alves Redol, 9, 1000-029 Lisbon, Portugal

November 2023

Abstract

One of the main performance-heavy areas in raytracing is light sampling. Light sampling is solved in a process called next-event estimation(NEE), where light samples are taken at each ray intersection. Since real-time rendering is an objective, instead of sampling all the luminaries, just a set of lights deemed more important are sampled by using a technique called Monte Carlo Importance Sampling. Aiming further acceleration, some importance sampling-based approaches build hierarchical data structures over all light sources, which results in high maintenance costs for dynamic scenes. This paper describes a two-level light bounding volume hierarchy (BVH) of mesh-based lights to accelerate light sampling while minimising the quality loss in dynamic scenes common in similar algorithms. Its main advantage is the ability to have dynamic lights without losing excessive accuracy or performance to maintenance operations. Our approach was developed as an extension to the Mogwai's PathTracer application from NVIDIA. The algorithm rebuilds the top-level structure in the CPU allowing it to retain its accuracy and refits only the bottom-level structures in need of updating on the GPU. The CPU rebuild is a relatively costly operation, to avoid excessive performance loss it is done asynchronously only being used in the next frame. We tested several quality metrics as well as frame times, our implementation achieved an up to 36% increase in MSE and 6% in SSIM with an average of 7% slowdown when compared with the single-level BVH.

Keywords: Ray Tracing, Bounding Volume Hierarchies, Vulkan Ray Tracing, Illumination, Acceleration Structures, Importance Sampling

1. Introduction

^a ^b

Raytracing has been used for years in offline rendering for movies, until recently it had proven overly demanding for real-time rendering. Now, with the hardware acceleration available in modern RTX GPUs real-time raytracing can be achieved. However, even with the added performance and acceleration of modern hardware we still need highly efficient algorithms to achieve real-time raytracing.

One of the main performance-heavy areas in raytracing is light sampling. Light sampling, usually solved with next event estimation, refers to the problem of calculating lighting for a surface from all light sources in the scene without considering reflections(indirect lighting). This is done by tracing rays to lights to see if the light is occluded by any object along the way, for complex scenes this

requires many ray intersections that are impossible to calculate in real-time as the number of rays to be traced scales linearly with the product of the number of lights, the resolution, the number of bounces, the number of samples per pixel.

Indirect lighting is another performance-heavy area in ray tracing, it is solved by tracing rays and bouncing them across the scene, at each collision we want to calculate the light contribution for the intersection point, so accelerating NEE will also accelerate indirect lighting.

Modern algorithms use importance sampling to severely reduce the amount of rays being traced. Importance sampling offers a trade-off between quality and performance depending on the amount of rays we choose to trace. One popular approach to importance sampling is through the use of tree data structures[11]. These usually have problematic upkeep costs but have proven to be an adequate solution for real-time rendering.

^a  <https://orcid.org/0009-0008-6672-5940>

^b  <https://orcid.org/0000-0002-8120-7649>

1.1. Objectives

The primary goal of this project was to create a two-level light bounding volume hierarchy (BVH). This BVH supports mesh-based lights and aims to speed up light sampling while reducing the quality loss typically seen in dynamic scenes, a common issue in similar algorithms. Our approach, based on Pierre Moreau [8] was developed as an extension to the Mogwai's [6] PathTracer application from NVIDIA. This pathtracer is one of the render graphs available in Mogwai. This application is based on Falcor [6], an open-source real-time rendering framework designed specifically for rapid prototyping to be used for research. We make use of the Pathtracer render graph and Vulkan ray tracing API to achieve realistic renders in real-time. We also wanted to confirm previously obtained results by testing against other methods available in Mogwai's Pathtracer render graph, these include the single-level variant of the BVH and the current best algorithm ReSTIR.

We tested the algorithms with a comprehensive set of metrics that have mostly not been used in previous research. These metrics aim to provide a more human-like evaluation of picture quality instead of the more simple mean square error(MSE).

The paper submission includes a concise video featuring a direct comparison between the single-level BVH provided by Mogwai and our implementation of a two-level BVH.

2. Background

2.1. Ray Tracing

Ray Tracing attempts to replicate how light works in real life by tracing rays across the scene to simulate how photons would bounce between objects before being picked by our eyes. If rays are traced from the light sources the likelihood of a ray hitting the camera is very low, instead, they are traced from the camera towards the scene.

The rays traced toward the scene originate in the camera position and go through a point in a viewport cell. The viewport is a 2D grid perpendicular to the camera direction, it is divided into cells, each representing a pixel in the rendered image.

In 1980 Whitted Ray tracing was introduced [11] providing a solution for indirect lighting, it creates a tree data structure by tracing a single path per pixel towards the scene and recursively generating new rays at collision with the scene's objects. The generated rays include reflected and refracted rays as well as rays directed at each light source. Whitted ray tracing can generate realistic images but it is extremely computationally demanding making it unusable for certain applications.

2.2. Real Time Rendering

Real-time rendering refers to the challenging task of rendering successive images fast enough that

the user cannot distinguish individual frames, it is usually used in interactive applications like games, where the user can input new commands that affect the rendering of the next frame at any time.

Until recently to render frames at a fast enough pace for real-time rendering the only technique capable of it was rasterization, however, games have now started implementing ray tracing as an optional graphical feature[1]. This is only possible due to algorithmic advances and NVIDIA's hardware acceleration.

2.3. Light Equation

The rendering equation introduced by Kajiya[5] is a mathematical formulation, to predict the intensity of light passing from a point x' in a surface to a point x , it takes into account intricate phenomena such as reflection, refraction, and shading, this equation enables us to calculate the appearance of objects with high accuracy and realism.

The equation is as follows:

$$I(x, x') = g(x, x') \left[\epsilon(x, x') + \int_S \rho(x, x', x'') I(x', x'') dx \right] \quad (1)$$

Where:

- $I(x, x')$, is related to the light intensity going from point x' to point x in $joule/m^4sec$, when x is a point in the camera viewport it gives the colour of a pixel.
- $g(x, x')$, is related to geometry, it is zero if the surface is occluded by another surface, if not it is $1/r^2$ where r is the distance from x to x' .
- $\epsilon(x, x')$, is related to the light intensity emitted from emissive surfaces from point x' to x .
- S , is the union of surfaces.
- $\rho(x, x', x'')$, is related to the light intensity scattered from an emissive point x'' to x by a patch of surface at x' .

The light equation is a useful tool for realistic rendering, however, solving the integral over S would require an infinite amount of samples. Monte Carlo Methods are used to solve quantitative problems in science through the use of statistical sampling[10]. In our case, we want to estimate $I(x, x')$ using a limited amount of samples acquired from traced rays.

The Monte Carlo method approximation can be improved without increasing the number of samples reducing performance, this can be done by using importance sampling, meaning we try to take more relevant samples to increase quality at the same sample count.

2.4. Mesh Light Sampling

To show shadows and direct lighting we need to create occlusion data rays called shadow rays, these are traced from the surface path being sampled to a point in each light.

When using mesh-based lights it is too computationally demanding to sample every triangle. Furthermore, in more complex ray tracing algorithms, direct lighting calculations are performed many times per pixel, making it also excessively demanding to try to sample every light.

This research focuses on importance sampling algorithms for NEE, mainly algorithms based on a BVH data structure. By sampling only a select few lights deemed more important good quality and performance can be obtained. This importance sampling technique is called next event estimation(NEE).

2.5. BVH Data Structure

The bounding volume hierarchy is a tree data structure where each node is associated with a volume of space and the objects contained within. The root node is associated with the entire scene, subsequent child nodes result from splitting the parent node's set of elements into smaller sets, one for each child node.

BVH's are better known in ray tracing for helping to accelerate ray-object intersections[2]. However, the BVH data structure implemented accelerates NEE.

When building BVHs for NEE, it is important to properly estimate each node's contributions. Alejandro Estevez and Christopher Kulla [4] proposed an algorithm to ensure that each node's importance is calculated with enough accuracy.

Additionally, Estevez and Kulla proposed a new heuristic for evaluating the quality of a node split. The new heuristic is called the surface area orientation heuristic(SAOH) it expands on the surface area heuristic by adding a light orientation component.

When using a BVH for NEE light sampling each node of the tree is a cluster of lights, usually emissive triangles but other kinds of lights are also supported. When choosing a light to sample, the BVH is sampled from top to bottom, at each level the contribution of each child node is estimated and used as its weight, the next node is a result of a weighted random choice of the child nodes. This way the PDF is estimated at sampling making it an appropriate estimation for each light/shaded surface sampled.

When the scene is animated the BVH structure needs to be updated, it can be updated with two operations. A rebuild operation that preserves quality but can't be run in real-time, and a refit operation where quality decays over time but is fast enough

for real-time rendering.

In this work we implement a novel approach using a two-level BVH introduced by P. Moreau et al. [8]. With a two-level BVH we can use both refits and rebuilds to achieve real-time rendering while preserving quality.

3. Implementation

3.1. Development Environment

NVIDIA's Mogwai is an open-source platform that allows for fast prototyping of algorithms, and its main feature is its modularity which further improves prototyping as parts of the rendering pipeline can be isolated in modules to simplify implementation.

Mogwai's modularity is achieved through the use of renderGraphs, which are chains of smaller programs called renderPasses each responsible for a set of features of the pipeline. The renderGraph in which we implemented our algorithm is the Path-Tracer renderGraph.

The PathTracer renderGraph is capable of running a simple unbiased path tracer great for creating ground truth images. It also already includes some NEE algorithms such as the single-level BVH in which our implementation is based, and ReStir [3] the current best algorithm for direct lighting.

3.2. Two-Level Light BVH

The two levels of the BVH divide the scene differently, the top-level acceleration structure(TLAS) divides the scene into groups of emissive meshes, each BLAS divides an emissive mesh into groups of emissive triangles. Each leaf node of the TLAS contains only one emissive mesh and a link to the corresponding bottom-level acceleration structure(BLAS) that will divide said emissive mesh, this structure is as seen in Figure 1.

By building the tree this way the refit operation only has to be applied to BLASes of changed mesh lights and the TLAS. When lights change TLAS and BLASes that need updating get refitted. The refitted TLAS is an asynchronously rebuilt TLAS from the previous frame.

Next, we describe each of the operations needed to use and maintain a two-level light BVH

3.3. BVH Building

Building the BVH is done in a top-down approach, this means we start at the root node and end at the leaves. In the case of a two-level BVH we start at the root of the TLAS, this node has an AABB that encompasses the entire scene, and a list of all lights.

Then we split the current node being processed into two child nodes, the parent AABB and light list are divided between the two child nodes. Finally, two new nodes are added to a stack of nodes, then we process the next node by picking it from the

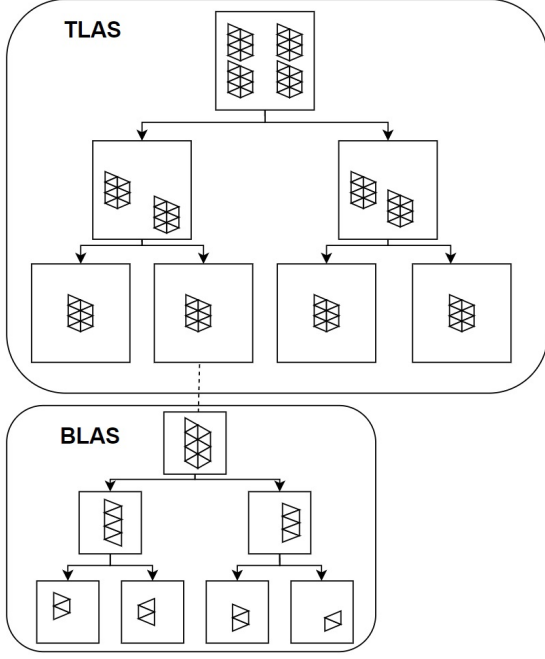


Figure 1: Two-level BVH structure. As can be seen the TLAS keeps the meshes intact and the BLASes subdivide meshes, the link between TLAS leaf and BLAS is also represented with a dotted line, for simplicity only one is shown but leaf nodes have a BLAS.

same node stack and repeating until we reach leaf nodes.

Once we reach a node with only one light we make it a leaf node and build a BLAS of the one light. The BLAS build is done in the same way as the TLAS, but instead of dividing a list of lights, it divides a list of emissive triangles. When the node being processed has fewer triangles than a certain threshold we make it a leaf node.

Building requires us to choose how to split a node's element list and AABB. To choose the plane that splits the nodes we use cost heuristics and binning.

Binning divides the element list into several sets of elements (bins). Then we calculate the cost of splitting between each of those bins and finally choose the lowest cost split.

To evaluate the cost of a split that divides the list of elements in a left group L and a right group R, we use one of two heuristics, these are SAH Eq.(2) which takes into consideration the number of triangles and their total area, and SAOH Eq.(3) which also considers the light's orientation yielding better picture quality.

$$cost_{SAH}(L, R) = \frac{n(L)a(L) + n(R)a(R)}{n(L \cup R)a(L \cup R)} \quad (2)$$

Where $n(G)$ and $a(G)$ refer respectively to the number of triangles in group G and their area.

$$cost_{SAOH}(L, R, s) = k_r(s) \frac{\Phi(L)a(L)M_\Omega(L) + \Phi(R)a(R)M_\Omega(R)}{a(L \cup R)M_\Omega(L \cup R)} \quad (3)$$

Where $k_r(s)$ is a regularization factor given by $k_r(s) = \frac{length_{max}}{length_s}$ this factor is used to penalize the choosing thin boxes and $M_\Omega(L)$ is an orientation based scalar.

Below, in Listing 1, we present pseudo code outlining the structure for building a BLAS. When building a TLAS, instead of dividing triangles, we would partition undivided meshlights. Additionally, when creating leaf nodes, we would also need to build the corresponding BLAS.

```

1
2 //Starts with list off all triangles
3 float BLASBuildInternal(){
4     Bin total = Bin();
5     For(i in all Triangles){
6         td = triangleData[i];
7         bins[getBinId(td)] |= td;
8     }
9     //compute costs with chosen heuristic
10    computeCosts();
11    //compute lowest cost
12    computeCheapestSplit();
13    if(split.isValid){
14        createInternalNode()
15        leftIndex=BLASBuildInternal()
16        RightIndex=BLASBuildInternal()
17    }else{
18        createLeafNode()
19    }
20 }

```

Listing 1: BLAS build function

3.4. BVH Refitting

Refitting is a much simpler operation than building, it is done on the GPU, the refit is done bottom-up as seen in Figure 2, processing the lowest level of BLASes first, then we refit the TLAS also starting at the lowest level. This means a node is refitted based on its child nodes. This implies we need to first process leaf nodes and only then their parent nodes.

Due to the fact that refitting is done inline, it needs to be highly efficient to avoid slowing down the render. This efficiency is achieved with GPU batching a process in which several GPU function calls are batched together to avoid delay overheads from multiple GPU calls.

Given these restrictions, the solution implemented is to first refit all BLASes and then the TLAS. All BLAS leaf nodes are batched into one GPU call followed by their parents into the next GPU call and so on until all BLAS nodes are refitted. Then we do the same for the TLAS.

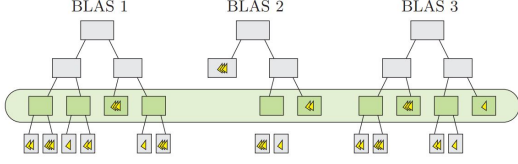


Figure 2: Figure form [8], showing all the BLASes being refit one dispatch per level with a bottom-up approach

To refit a BLAS leaf node we iterate over all of its triangles and calculate its new AABB and normal bounding cone. Internal nodes have their AABB and normal bounding cones adjusted to encompass their child nodes, and TLAS leaf nodes copy the corresponding BLAS root node properties.

3.5. BVH TLAS Rebuilding

The rebuilding of the TLAS happens in the same way as the building, but when a leaf node is created instead of building a BLAS it links to the correct already built BLAS.

Rebuilding happens asynchronously to minimize performance costs. With asynchronous processing comes a synchronisation problem.

We want to have as much time as possible to rebuild so we want the earliest possible location to start the operation. Since the rebuild uses data created during the TLAS refit, the earliest moment in which we can launch the rebuild is immediately after.

To maximize how much time there is to rebuild we must delay the final synchronisation for as long as possible, which corresponds to allowing it to start syncing at the beginning of the next frame.

This final synchronization has to finish before the refit operation of the next frame, waiting if needed.

3.6. BVH Sampling

To generate a light sample from a two-level light BVH we traverse the BVH starting at the TLAS root until we reach a BLAS leaf.

To navigate the BVH (Bounding Volume Hierarchy), a random number is initially generated. This number determines which branch of the tree to explore. At each node, the random number is compared to the probability of selecting each branch.

The probability of choosing each node varies with their importance. This importance can be measured with the combination of the following metrics, distance, light flux, light orientation and light visibility.

Combining these metrics the following expression

is used to calculate each node's importance:

$$importance(X, C) = \frac{\Phi(C) |\cos \theta'_i|}{\|X - C\|^2} xg(\theta') \quad (4)$$

Where X is the shading point and C is the center of the AABB, $g(\theta')$ is $\cos \theta'$ if $\theta' < \theta_e$ and zero otherwise, $\theta'_i = \max(0, \theta_i - \theta_u)$ and $\theta' = \max(0, \theta - \theta_o - \theta_u)$ where θ_i is the incident angle and θ_u is the uncertainty angle for the bounding cone to cover all emissive triangles, all these can be found in Figure 3.

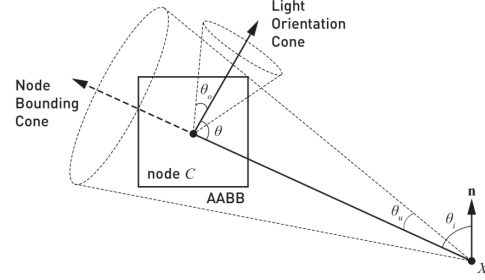


Figure 3: Image taken from [8]. Geometry used for computing the importance of each node.

4. Experiments

To evaluate the performance of the developed algorithm we tested it against its single-level variant and ReStir. This way we can study the advantages of rebuilding the top structure as well as the consequences of restricting splitting to entire meshes in the top structure. We also compared the quality of the SAH and SAOH splitting heuristics with the two-level BVH.

The rendered images were tested against ground truth images we generated with Mogwais Path-Tracer.

We used two scenes to test the algorithms. The first, NVIDIA's Bistro [7] seen in figure 4, features a long street allowing us to showcase quality decay over long stretches of movement. The other test scene was Emerald Square [9] as seen in figure 5, it offers a more expansive environment filled with numerous lights, creating a complex lighting scenario for analysis.

4.1. Metrics

The metrics tested include MSE since it is used in previous research and serves as a point of comparison, SSIM and PSNR are metrics that aim to evaluate a more human perspective on image quality.

Mean Square Error (MSE) is a measure of the average squared difference between each pixel pair of two images.

Unlike MSE Structural Similarity (SSIM) attempts to compare the differences in structure, con-



Figure 4: Ground truth image used for Bistro test scene at frame 150.



Figure 5: Ground truth image used for Emerald Square test scene at frame 150.

trast and luminance, this provides a metric that is closer to a human’s sense of the difference between two pictures. It takes a value between zero and one.

Peak Signal-To-Noise Ratio is a metric that quantifies signal noise which is often generated by importance sampling of direct light making it an important metric to test.

To evaluate performance we measured total frame time. Regarding both the rebuilding and refitting impacts, we measured the time to completion of each task.

4.2. Results

In the Table 1 we have the averages over the several samples tested for each frame. Then in Figure 6 we show the graphs that map those averages shown in Table 1. We can see a tendency as the animation evolves, the metrics get worse over time due to not having a full rebuild. The better updating from the two-level BVH mitigates some of the quality loss.

When comparing the single-level BVH to the two-level BVH we can see there is a significant difference between the two, the two-level BVH seems to outperform the single-level in MSE by 17% on average with an up to 36% improvement and in SSIM by 6% on average.

Bistro Exterior					
SingleLevel Average Frames:	0	50	100	150	Average
MSE	460,76	516,73	515,09	846,48	584,76
SSIM	0,525	0,500	0,509	0,462	0,499
PSNR	23,253	22,628	22,800	21,263	22,486
TwoLevel SAOH Average Frames:					
MSE	426,26	440,77	438,56	688,05	498,41
SSIM	0,542	0,538	0,538	0,493	0,53
PSNR	23,583	23,462	23,591	22,407	23,26

Table 1: Table with the average results for all metrics in tested frames of our version of Bistro.

Emerald Square								
SingleLevel Average Frames:	0	50	100	150	200	250	300	Average
MSE	866,58	958,80	1000,34	1058,23	1082,13	1047,20	1127,70	1020,14
SSIM	0,621	0,611	0,617	0,626	0,620	0,627	0,617	0,620
PSNR	22,028	21,548	21,185	20,932	20,423	20,669	20,216	21,000
TwoLevel SAH Average Frames:								
MSE	874,51	1009,89	1046,95	1147,70	1185,29	1191,35	1235,20	1098,70
SSIM	0,622	0,619	0,622	0,628	0,613	0,618	0,609	0,62
PSNR	22,032	21,297	20,810	20,359	19,840	20,152	19,775	20,61
TwoLevel SAOH Average Frames:								
MSE	857,70	984,79	1035,97	1060,25	1064,31	1064,95	1107,94	1025,13
SSIM	0,625	0,623	0,620	0,636	0,632	0,635	0,634	0,63
PSNR	22,037	21,419	20,837	20,925	20,636	20,738	20,682	21,04
ReStIR Average Frames:								
MSE	488,41	567,45	655,15	677,23	644,14	685,30	669,02	626,67
SSIM	0,684	0,674	0,673	0,687	0,693	0,682	0,684	0,68
PSNR	22,233	21,657	21,014	20,924	21,138	20,904	20,953	21,26

Table 2: Table with the average results for all metrics in tested frames of our version of Emerald Square.

The results obtained in Emerald Square in Table 2 show minimal difference between the single-level and two-level BVH, we can also observe the SAOH metric outperforming the SAH metric. The ReStir results were considerably better than the other algorithms with a 60% improvement in MSE and 8% in the SSIM metric. When looking at the close-up images of Emerald Square provided in Figure 7, we see much better results for the two-level SAH and SAOH pictures than for the single-level image, at the beginning of the animation there was a bus in this location, the two-level BVH can rebuild the top structure making the number of samples taken of the bus much lower after the bus left, the single-level BVH keeps trying to sample the bus that no longer illuminates that place resulting in lower quality and many black pixels from missed shadow rays.

In Table 3 we have the frame time results for the single-level BVH and two-level BVH both using SAOH. These results were obtained in a PC with an Intel i5-4690k at stock frequency and an NVIDIA RTX 2080. The results are averages over all frames of animation with a simulated 25fps.

Algorithm:	Measure One		Emerald Square		Bistro Exterior	
	Single Level BVH	Two Level BVH	Single Level BVH	Two Level BVH	Single Level BVH	Two Level BVH
BVH Refit (ms)	4	7	7	9,2	4,2	6,3
BVH Rebuild (ms)	7,2	..	..	12,5	..	3,2
Frame Time (ms)	61(16,4FPS)	66(15,2FPS)	60(16,7FPS)	63(15,8FPS)	36(27,7FPS)	38(26,3FPS)

Table 3: Performance Results

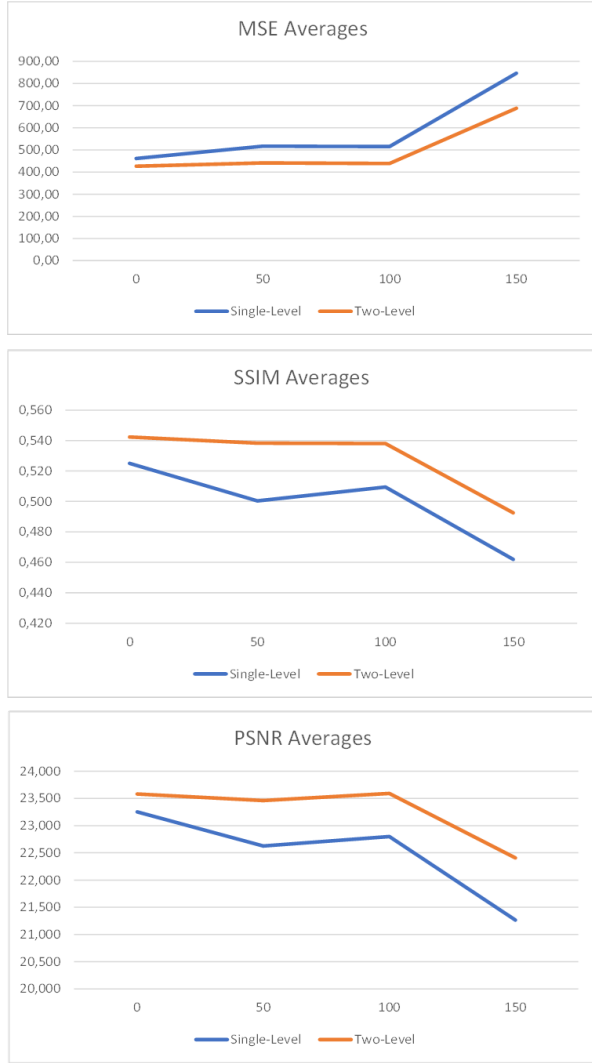


Figure 6: Graphs mapping the averages of each metric from Table 1.

Overall the studied metrics showed the expected improvements and were close to what was reported by the work described in [8]. We can see from the close picture that the two-level BVH is working as expected improving quality in areas affected by moving lights.

Performance was as expected the top-level refit takes too much time to compute for inline execution but as it is done asynchronously it almost doesn't affect performance. In total frame times, we saw an average of 7% slow down which is consistent with what was reported in [8]

5. Conclusions

The results obtained in the Emerald scene showed no improvement of metrics. However, when examining picture quality from zoomed-in pictures we saw more detail in certain areas with the two-level BVH compared to the single-level BVH; the opposite was

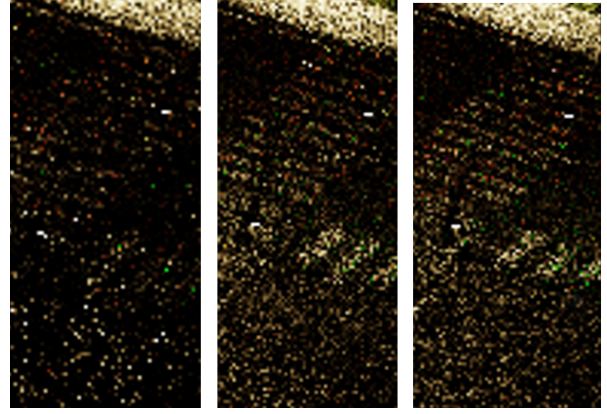


Figure 7: Zoomed in pictures of frame 150 of our version of Emerald Square with four samples accumulated. On the left using the Light BVH, in the middle using the two-level with SAH and on the right the two-level BVH with SAOH. It shows the initial position of one of the animated buses.

also true but to a lesser extent. In our Bistro scene testing, we obtained positive results with up to 36% improvements in MSE and 6% in SSIM.

Performance was as expected the added processing from the TLAS rebuild had little to no effect on performance due to its asynchronous processing but the refit was slightly slower.

When testing ReSTIR we saw an overall improvement in quality as was expected from the current best solution.

ReSTIR is the best solution of the three tested for all scenarios.

The two-level BVH showed some improvements over the single-level version, especially evident in scenarios where lights had undergone substantial updates. The improvements happen in regions influenced by both the current and initial positions of the lights.

Acknowledgements

The work reported in this article was partially supported under the auspices of the UNESCO Chair on AI & VR by national funds through Fundação para a Ciência e a Tecnologia with references DOI:10.54499/UIDB/-50021/2020, DOI:10.54499/DL57/2016/CP1368/-CT0002 and 2022.09212.PTDC (XAVIER project).

The work was also carried on under DL 60/2018, 3-08 – I&D activity at INESC -ID, project PEST 2020 -GI (graphics & interaction (UIDB/50021/2020)).

References

- [1] Cyberpunk 2077 overdrive benchmarks: Melt your eyeballs and gpu — pcworld.

<https://www.pcworld.com/article/1782490/cyberpunk-2077-overdrive-mode-benchmarks-nvidia-rtx-path-tracing.html>. (Accessed on 07/25/2023).

- [2] G. v. d. Bergen. Efficient collision detection of complex deformable models using aabb trees. *Journal of graphics tools*, 2(4):1–13, 1997.
- [3] B. Bitterli, C. Wyman, M. Pharr, P. Shirley, A. Lefohn, and W. Jarosz. Spatiotemporal reservoir resampling for real-time ray tracing with dynamic direct lighting. *ACM Transactions on Graphics (TOG)*, 39(4):148–1, 2020.
- [4] A. Conty Estevez and C. Kulla. Importance sampling of many lights with adaptive tree splitting. *Proceedings of the ACM on Computer Graphics and Interactive Techniques*, 1(2):1–17, 2018.
- [5] J. T. Kajiya. The rendering equation. In *Proceedings of the 13th annual conference on Computer graphics and interactive techniques*, pages 143–150, 1986.
- [6] S. Kallweit, P. Clarberg, C. Kolb, T. Davidovič, K.-H. Yao, T. Foley, Y. He, L. Wu, L. Chen, T. Akenine-Möller, C. Wyman, C. Crassin, and N. Benty. The Falcor rendering framework, 8 2022. <https://github.com/NVIDIAGameWorks/Falcor>.
- [7] A. Lumberyard. Amazon lumberyard bistro, open research content archive (orca), July 2017. <http://developer.nvidia.com/orca/amazon-lumberyard-bistro>.
- [8] P. Moreau, M. Pharr, and P. Clarberg. Dynamic many-light sampling for real-time ray tracing. In *High Performance Graphics (Short Papers)*, pages 21–26, 2019.
- [9] K. A. Nicholas Hull and N. Benty. Nvidia emerald square, open research content archive (orca), July 2017. <http://developer.nvidia.com/orca/nvidia-emerald-square>.
- [10] M. Pharr, W. Jakob, and G. Humphreys. *Physically based rendering: From theory to implementation*. MIT Press, 2023.
- [11] T. Whitted. An improved illumination model for shaded display. *Communications of the ACM*, 23(6):343–349, June 1980.